REPORTE DOCKER

RICARDO GADIEL BOBADILLA GABRIEL

Introducción

Este manual presenta el proceso de instalación y configuración de un entorno integrado por **Docker Desktop**, **PostgreSQL** y **pgAdmin 4** en un equipo con sistema operativo Windows.

En lugar de realizar una instalación directa de PostgreSQL en el sistema operativo, este gestor de bases de datos se ejecuta dentro de un contenedor de Docker. De manera independiente, **pgAdmin 4** se ejecuta en su propio contenedor, estableciendo comunicación con PostgreSQL a través de una red virtual de Docker configurada específicamente para este fin.

Para comprender la arquitectura desde el principio, es importante distinguir los siguientes componentes:

- **Docker Desktop:** Aplicación principal para utilizar Docker en entornos Windows.
- **Docker Engine:** Motor subyacente encargado de ejecutar y administrar los contenedores.
- **WSL 2 (Windows Subsystem for Linux 2):** Entorno de backend que utiliza Docker Desktop en Windows para optimizar el rendimiento y la ejecución de los contenedores.

El manual está estructurado en dos partes:

1. **Parte 1:** Tiene como objetivo verificar que todos los elementos de la infraestructura funcionen adecuadamente (Docker, WSL 2, imágenes, contenedores, puertos, volúmenes y redes virtuales), concluyendo con el registro exitoso del servidor PostgreSQL dentro de pgAdmin.
2. **Parte 2:** Se enfocará en documentar la estructura de la base de datos implementada, incluyendo sus tablas, esquemas, usuarios, roles, permisos y consultas de prueba.

Conceptos Fundamentales

Antes de comenzar con la instalación es conveniente identificar la función de cada componente que forma parte del entorno. Aunque Docker, PostgreSQL y pgAdmin 4 trabajan conjuntamente, cada herramienta cumple una función diferente.

¿Qué es Docker?

Es una plataforma que permite empaquetar, crear y ejecutar aplicaciones dentro de **contenedores**. Un contenedor proporciona un entorno virtualizado y aislado donde una aplicación corre junto con todas sus dependencias necesarias.

En este manual, Docker Desktop se utiliza para administrar y aislar los servicios de PostgreSQL y pgAdmin 4 mediante una interfaz gráfica y comandos de terminal, evitando configuraciones complejas directamente en el sistema operativo anfitrión.

¿Qué es PostgreSQL?

Es un sistema gestor de bases de datos relacional (SGBDR) de código abierto, robusto y altamente escalable, que permite almacenar, organizar y consultar grandes volúmenes de información mediante el lenguaje estándar SQL.

¿Qué es pgAdmin 4?

pgAdmin 4 es una herramienta gráfica utilizada para administrar servidores y bases de datos PostgreSQL. Su interfaz permite realizar diferentes tareas sin depender exclusivamente de la terminal, como conectarse a un servidor, consultar bases de datos, ejecutar instrucciones SQL y administrar diferentes elementos del sistema gestor.

En el entorno propuesto en este manual, pgAdmin 4 se instala directamente en Windows (Parte 1) o se ejecuta como contenedor independiente (Parte 2), y posteriormente se configura para comunicarse con el servidor PostgreSQL que se encuentra ejecutándose dentro de Docker.

Relación entre Docker, PostgreSQL y pgAdmin 4

Las tres herramientas cumplen funciones diferentes, pero trabajan juntas dentro del entorno. La relación puede representarse de la siguiente manera:

text

Windows → Docker Desktop → Contenedor PostgreSQL → pgAdmin 4

- **Windows:** sistema operativo donde se prepara el entorno.
- **Docker Desktop:** administra los contenedores.
- **PostgreSQL:** funciona dentro del contenedor y proporciona el servicio de base de datos.
- **pgAdmin 4:** se conecta al servidor PostgreSQL para permitir su administración mediante una interfaz gráfica.

Por lo tanto, pgAdmin 4 no reemplaza a PostgreSQL y Docker tampoco funciona como gestor de bases de datos. Cada componente cumple una función específica dentro del entorno.

Requisitos previos

- Windows 10 de 64 bits, en una edición y compilación compatibles con la versión actual de Docker Desktop, o Windows 11 de 64 bits.
- Virtualización de hardware habilitada en el BIOS/UEFI del equipo (Intel VT-x o AMD-V).
- Se recomienda contar con 8 GB de RAM o más.
- Espacio libre en disco suficiente para Docker Desktop, las imágenes descargadas y los datos de PostgreSQL.
- Cuenta de Windows con permisos de administrador.
- Conexión a Internet, ya que se descargarán instaladores e imágenes de varios cientos de megabytes.

Instalación de Docker Desktop

Docker es una plataforma que permite ejecutar aplicaciones mediante contenedores, los cuales son entornos independientes y ligeros que reúnen los elementos necesarios para el funcionamiento de un programa. De esta forma, es posible utilizar diferentes aplicaciones sin tener que instalar todos sus

componentes directamente en Windows. Para este manual, Docker se empleará como base para ejecutar PostgreSQL y pgAdmin sin realizar una instalación convencional de estos programas.

Descarga de Docker Desktop

Paso 1. Abrir un navegador web y acceder al sitio oficial de Docker: Docker Desktop

Paso 2. Seleccionar la opción **"Download Docker Desktop"** y posteriormente elegir la versión correspondiente al sistema operativo Windows.

Paso 3. Esperar hasta que finalice la descarga del archivo de instalación.

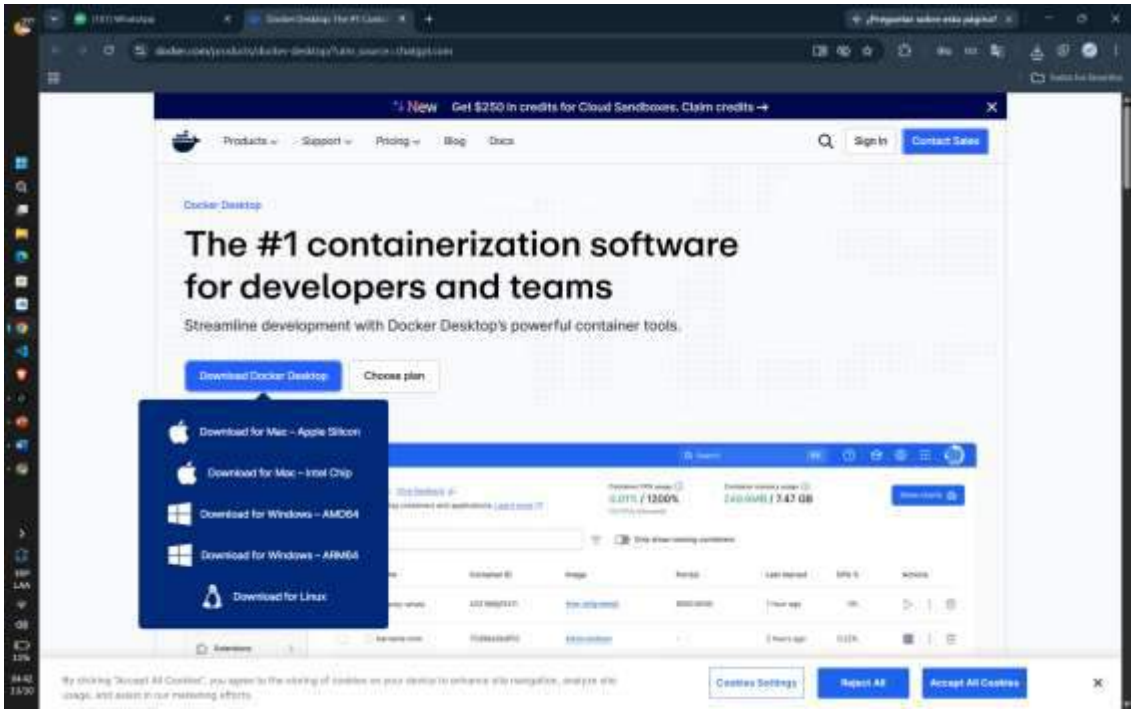


Imagen 1 Página oficial de Docker donde se muestran las opciones disponibles para descargar Docker Desktop.

Instalación de Docker Desktop

Paso1. Localizar y ejecutar el archivo descargado **"Docker Desktop Installer"**. Si Windows muestra el aviso de Control de cuentas de usuario (UAC), seleccionar la opción correspondiente para permitir la ejecución.

Paso 2. En la ventana denominada **"Configuration"**, se muestran las opciones iniciales de instalación. De manera predeterminada, aparece seleccionada **"Per-user installation (Recommended)"** y se encuentra habilitada la opción **"Use WSL 2 instead of Hyper-V"**.

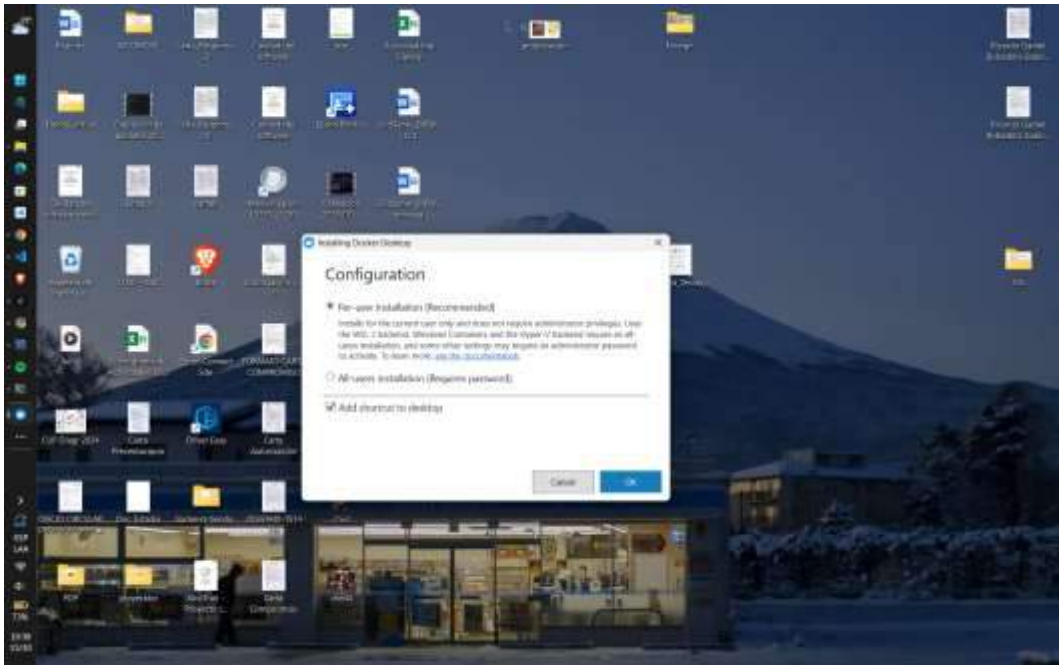


Imagen 2 Ventana de configuración del instalador con la opción "Per-user installation" seleccionada de forma predeterminada.

Paso 3. Para este procedimiento, se seleccionó **"All-users installation (Requires password)"**, manteniendo activa la opción **"Use WSL 2 instead of Hyper-V"**. Después, se hizo clic en **"OK"** para continuar con la instalación.

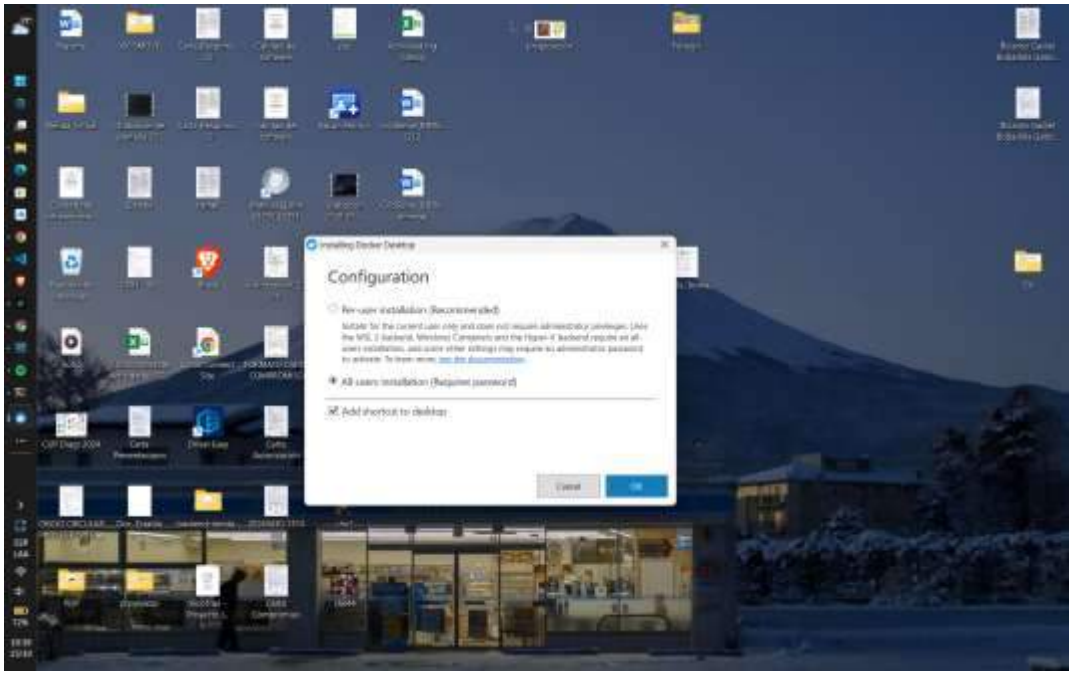


Imagen 3 Configuración empleada para la instalación: disponibilidad para todos los usuarios y uso de WSL 2 como backend.

Paso 4. Esperar mientras el instalador realiza la copia de archivos y configura los componentes necesarios para el funcionamiento de Docker Desktop. En caso de que el sistema lo solicite, reiniciar el equipo para completar el proceso.

Verificación de WSL 2 y Docker

WSL 2 es el componente que permite ejecutar un kernel de Linux dentro de Windows, y es el backend que Docker Desktop utiliza en este manual.

Verificación e instalación de WSL 2

Paso 1. Abrir una terminal como administrador y ejecutar:

```
wsl --version
wsl --status
wsl --update
```

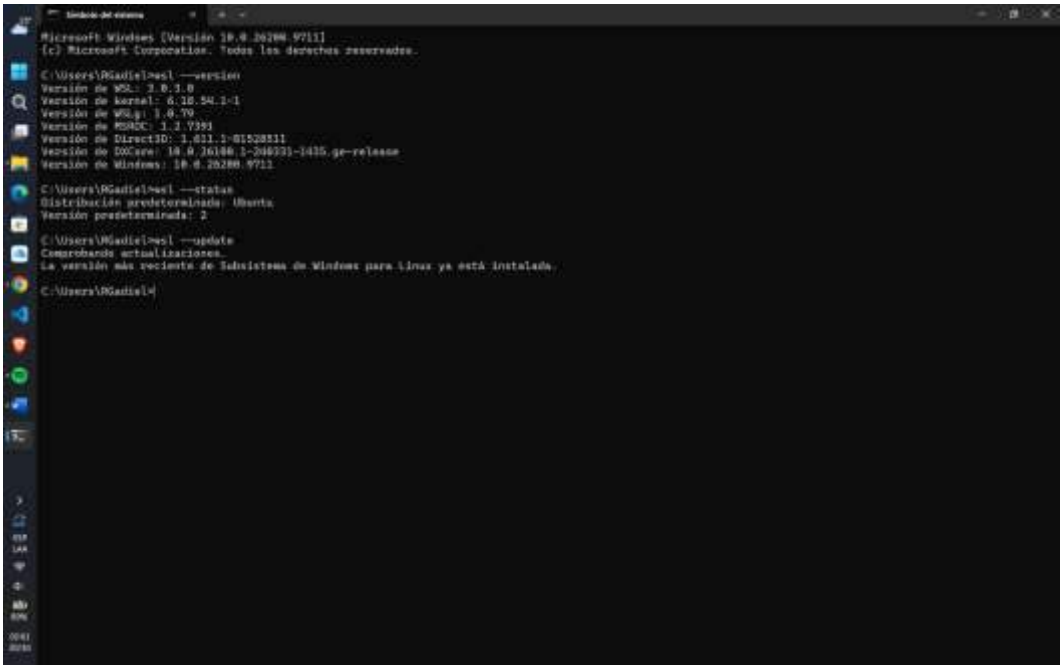


Imagen 4 Verificación del estado de WSL desde la terminal (wsl --version, --status, --update)

- **Qué debe observarse:** La versión de WSL (2.7.13.0), del kernel, de WSLg y de Windows; el estado indicando que WSL1 no es compatible con la configuración actual del equipo; y la actualización del Subsistema de Windows para Linux a la versión 2.7.14.

Paso 2. En mi caso ya tenengo una distribución de Linux instalada en mi sistema y en caso de que todavía no tenía una distribución Linux instalada, se instaló Ubuntu:

```
wsl --install
```

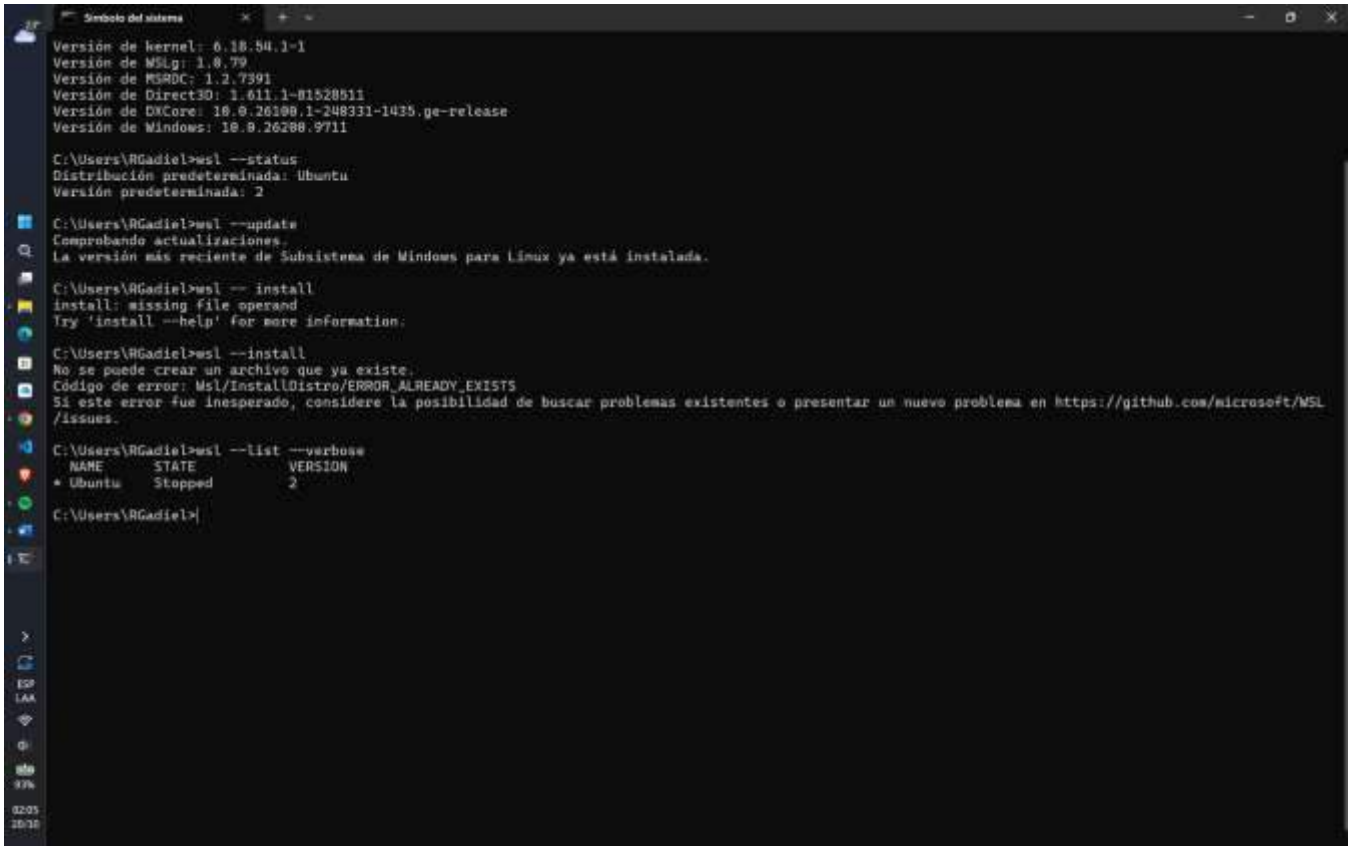


Imagen 5 Instalación de la distribución Ubuntu mediante wsl --install

- **Qué debe observarse:** La descarga e instalación de Ubuntu, la creación de la cuenta de usuario Unix "silverwolf" y su contraseña, y el mensaje final ubicando la terminal dentro de la distribución instalada.
- **Resultado esperado:** WSL 2 queda con una distribución Linux (Ubuntu) instalada y funcional, lista para que Docker Desktop la utilice como backend

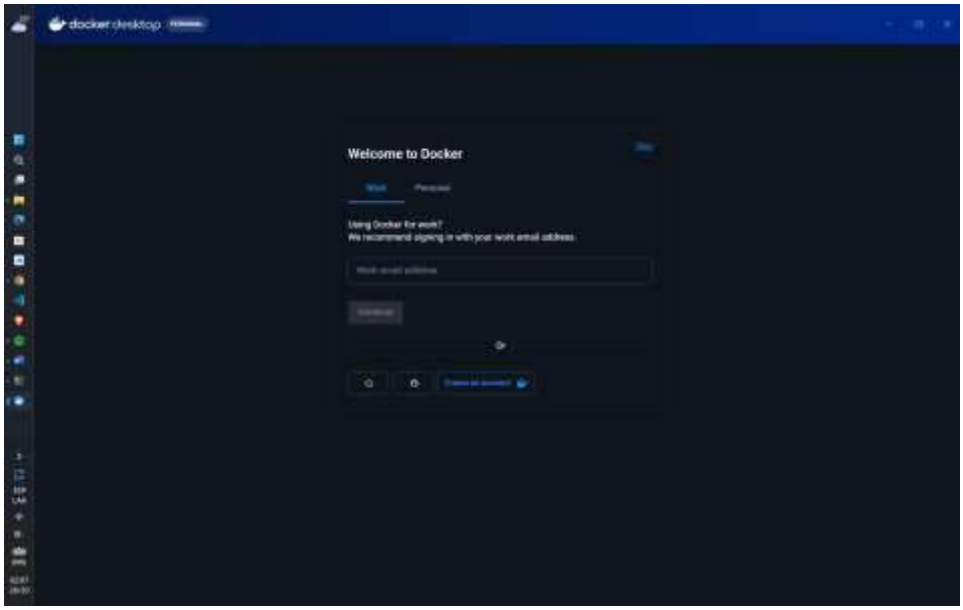
Primer inicio de Docker Desktop

Al abrir Docker Desktop por primera vez después de la instalación, la aplicación solicitó aceptar el acuerdo de suscripción:



Figura 6. Acuerdo de suscripción de Docker Desktop (Docker Subscription Service Agreement)

A continuación, se presentó una pantalla de bienvenida con opciones para iniciar sesión (con cuenta de trabajo, personal, Google, GitHub o creando una cuenta nueva) o bien continuar sin iniciar sesión mediante la opción "Skip":



Una vez completada esta configuración inicial, Docker Desktop mostró el panel de "Containers" vacío, con el motor (Engine) activo:

Imagen 6 Pantalla de bienvenida de Docker Desktop con las opciones de inicio de sesión.

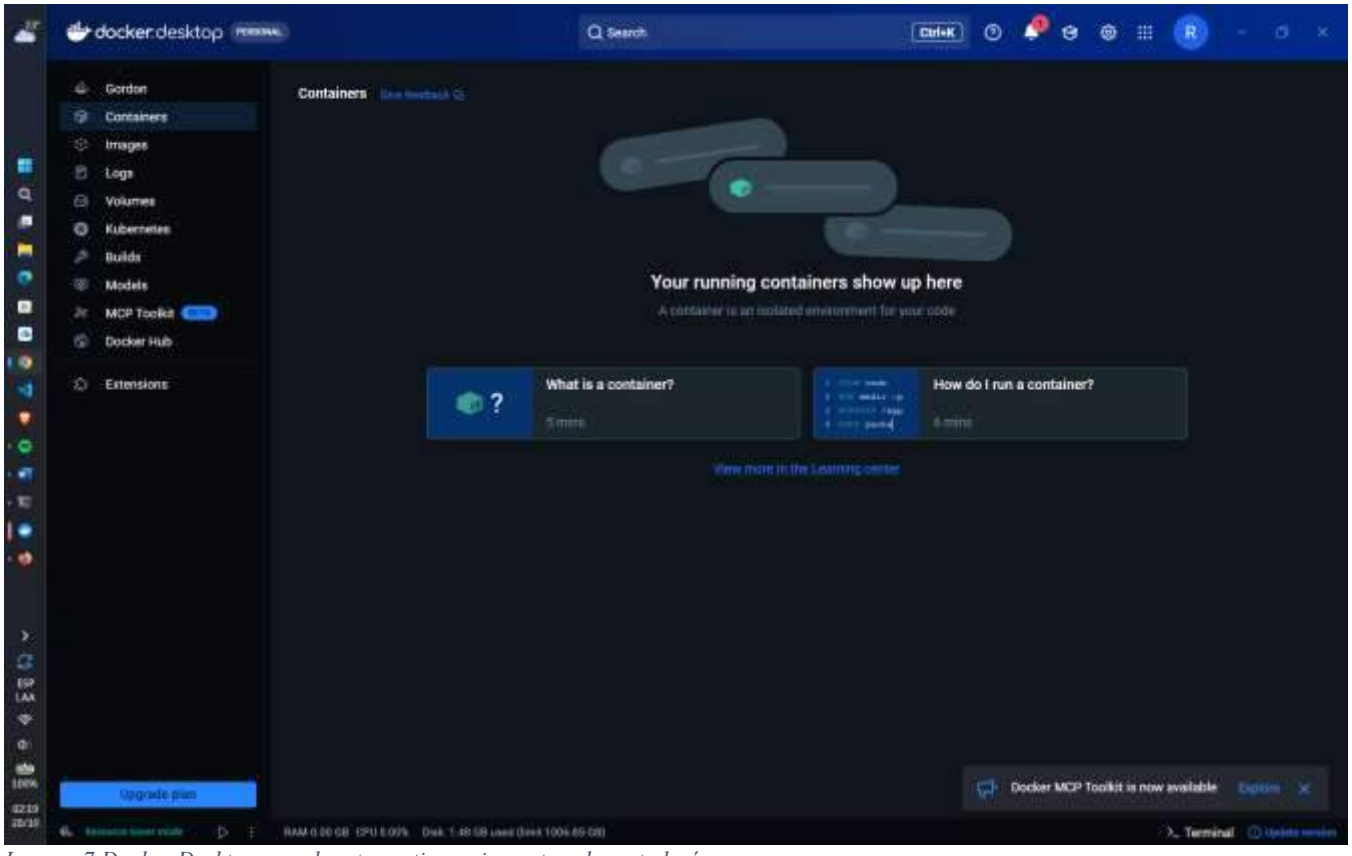


Imagen 7 Docker Desktop con el motor activo y sin contenedores todavía

Verificación desde la terminal

Paso 1. Confirmar que Docker Desktop aparece instalado como aplicación en Windows y que el cliente de Docker responde desde la terminal:

```
docker --version
```

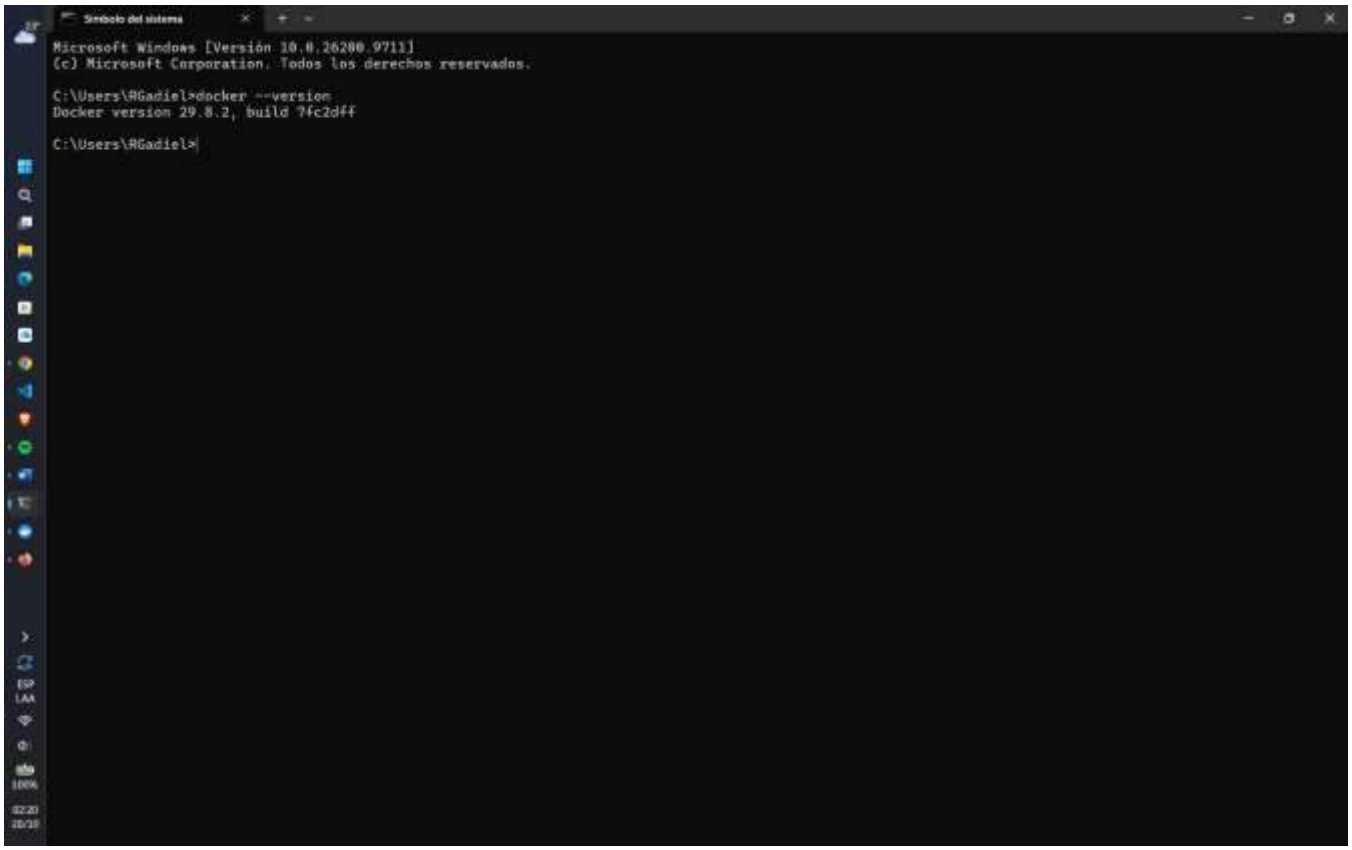


Imagen 8 comando docker --version

Paso 2. Confirmar que Docker puede ejecutar contenedores, con la imagen de prueba oficial:

```
docker run hello-world
```

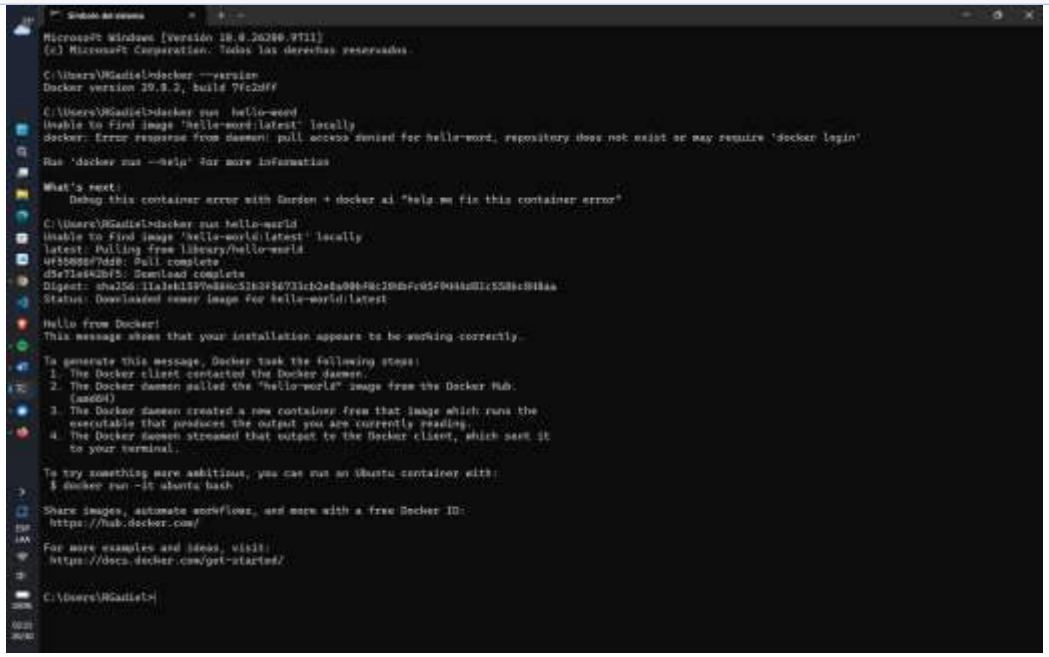


Imagen 9 Ejecución de docker run hello-world

Descarga de PostgreSQL

PostgreSQL se ejecuta de forma aislada dentro del entorno de Docker, sin instalarse directamente sobre Windows.

Versión utilizada

La imagen oficial de PostgreSQL en Docker Hub ofrece varias etiquetas, entre ellas "postgres:latest". Este manual fija una versión específica, postgres:16, para que el procedimiento sea reproducible.

Descarga de la imagen

```
docker pull postgres:16
docker images
```

Paso 1. Con Docker Desktop en ejecución, descargar la imagen y confirmar que quedó disponible localmente

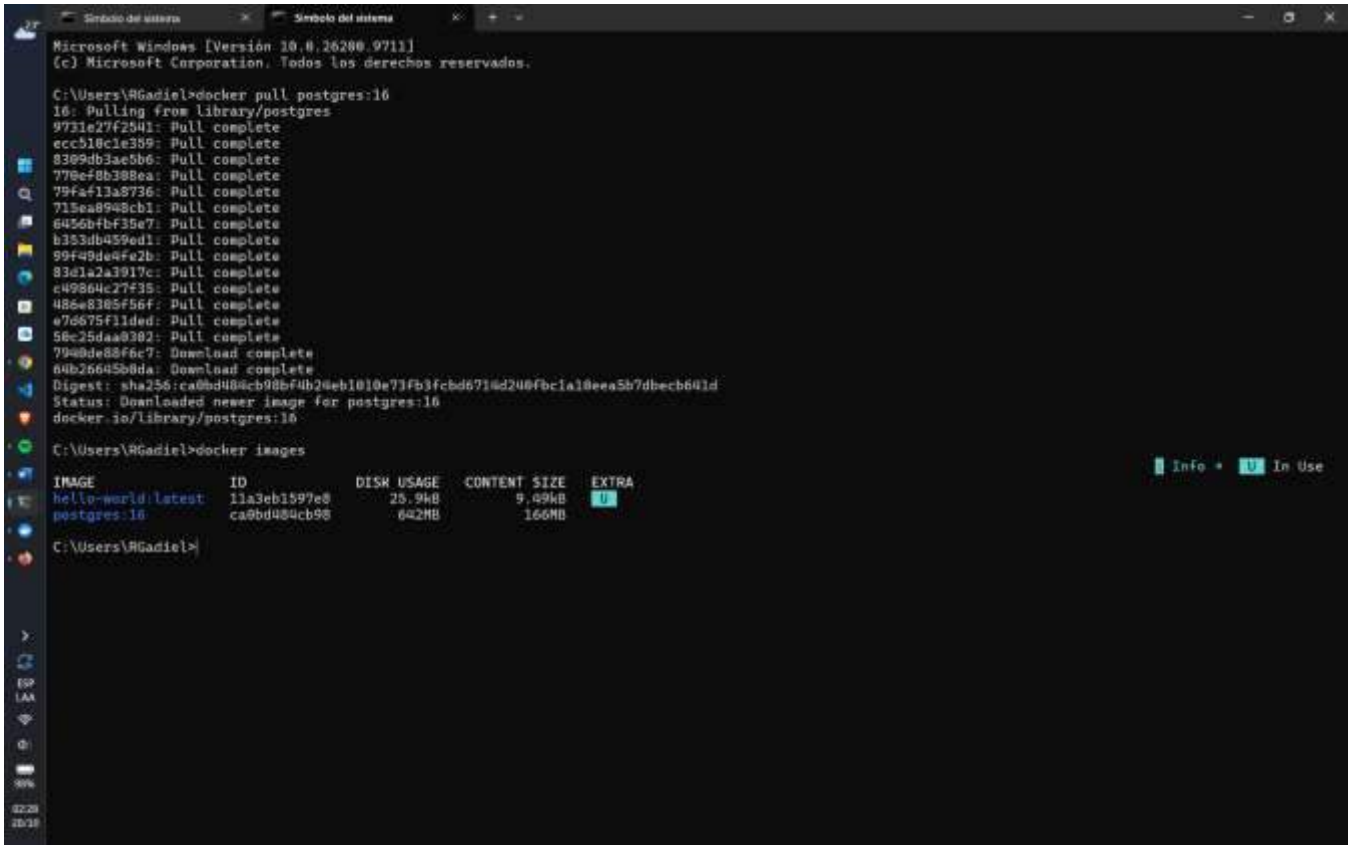


Imagen 10 Descarga de la imagen postgres:16 y comprobación con docker images

- **Qué debe observarse:** La descarga por capas de "postgres:16" y, en "docker images", las imágenes "hello-world:latest" (25.9 kB) y "postgres:16" (642 MB) disponibles localmente.
- **Resultado esperado:** La imagen postgres:16 aparece disponible localmente, junto con hello-world:latest

Creación del contenedor PostgreSQL

Creación mediante docker run

El contenedor se creó desde el Símbolo del sistema (CMD), dividiendo el comando en varias líneas mediante el carácter de continuación "^" para facilitar su lectura:

```
docker run ^
  --name postgres-db ^
  -e POSTGRES_USER=admin ^
  -e POSTGRES_PASSWORD=admin123 ^
  -e POSTGRES_DB=mi_base_datos ^
```

El símbolo "^" al final de cada línea es el carácter de continuación de línea en CMD (en PowerShell el equivalente es la comilla invertida " ` "); permite escribir el mismo comando de forma más legible sin afectar su funcionamiento. A continuación, se explica cada parámetro:

- --name postgres-db: asigna el nombre "postgres-db" al contenedor, para identificarlo en comandos posteriores.
- -e POSTGRES_USER=admin: configura "admin" como superusuario de PostgreSQL dentro del contenedor.
- -e POSTGRES_PASSWORD=admin123: contraseña de ese usuario. Es un valor de demostración para un entorno local; no debe usarse en producción.
- -e POSTGRES_DB=mi_base_datos: nombre de la base de datos creada automáticamente al inicializar el volumen.
- -p 5432:5432: mapea el puerto 5432 de Windows con el puerto 5432 del contenedor.
- -v postgres-data:/var/lib/postgresql/data: crea o reutiliza el volumen "postgres-data", que conserva los datos aunque el contenedor se detenga o se elimine.
- -d: ejecuta el contenedor en segundo plano.
- postgres:16: imagen a partir de la cual se crea el contenedor.

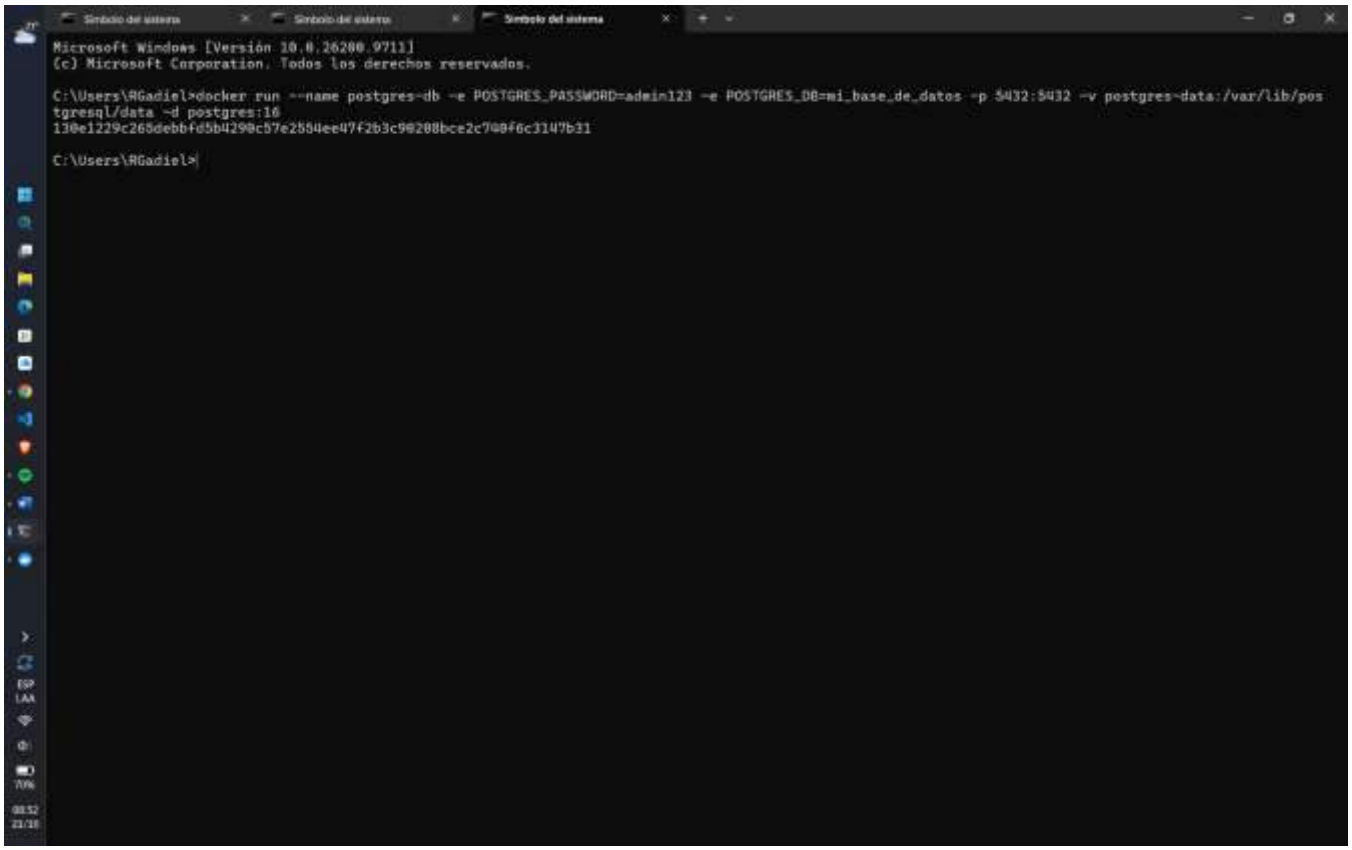


Imagen 11 Creación del contenedor postgres-db mediante docker run

- **Qué debe observarse:** El identificador alfanumérico devuelto por Docker (130e1229c265debbfd5b4290c57e2554ee47f2b3c90208bce2c740f6c3147b31), confirmando que el contenedor se creó correctamente.
- **Resultado esperado:** El contenedor se crea sin errores y devuelve su identificador.

Verificación del volumen

El volumen postgres-data, montado en /var/lib/postgresql/data dentro del contenedor, permite conservar los datos de PostgreSQL aunque el contenedor se detenga o se elimine. Conviene comprobar su existencia y sus metadatos:

```
docker volume ls

docker volume inspect postgres-data
```

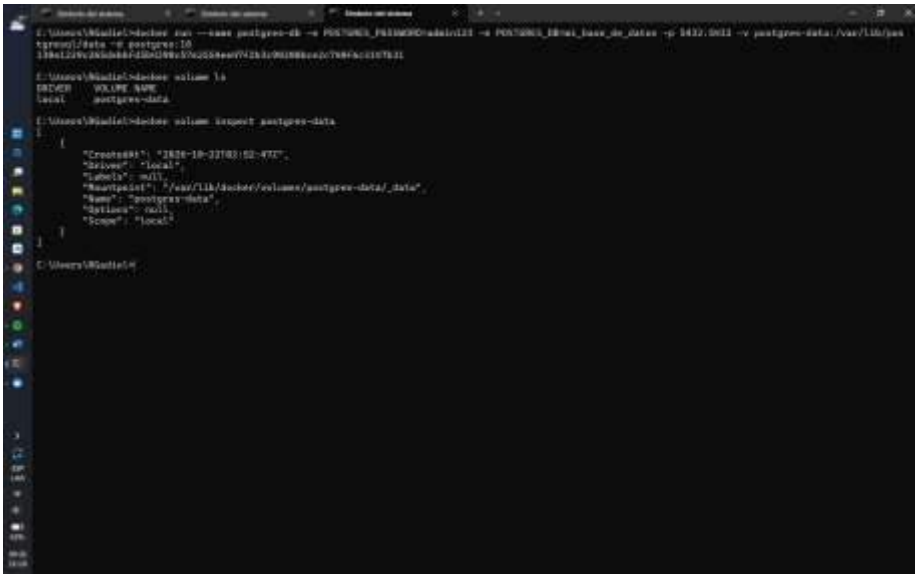


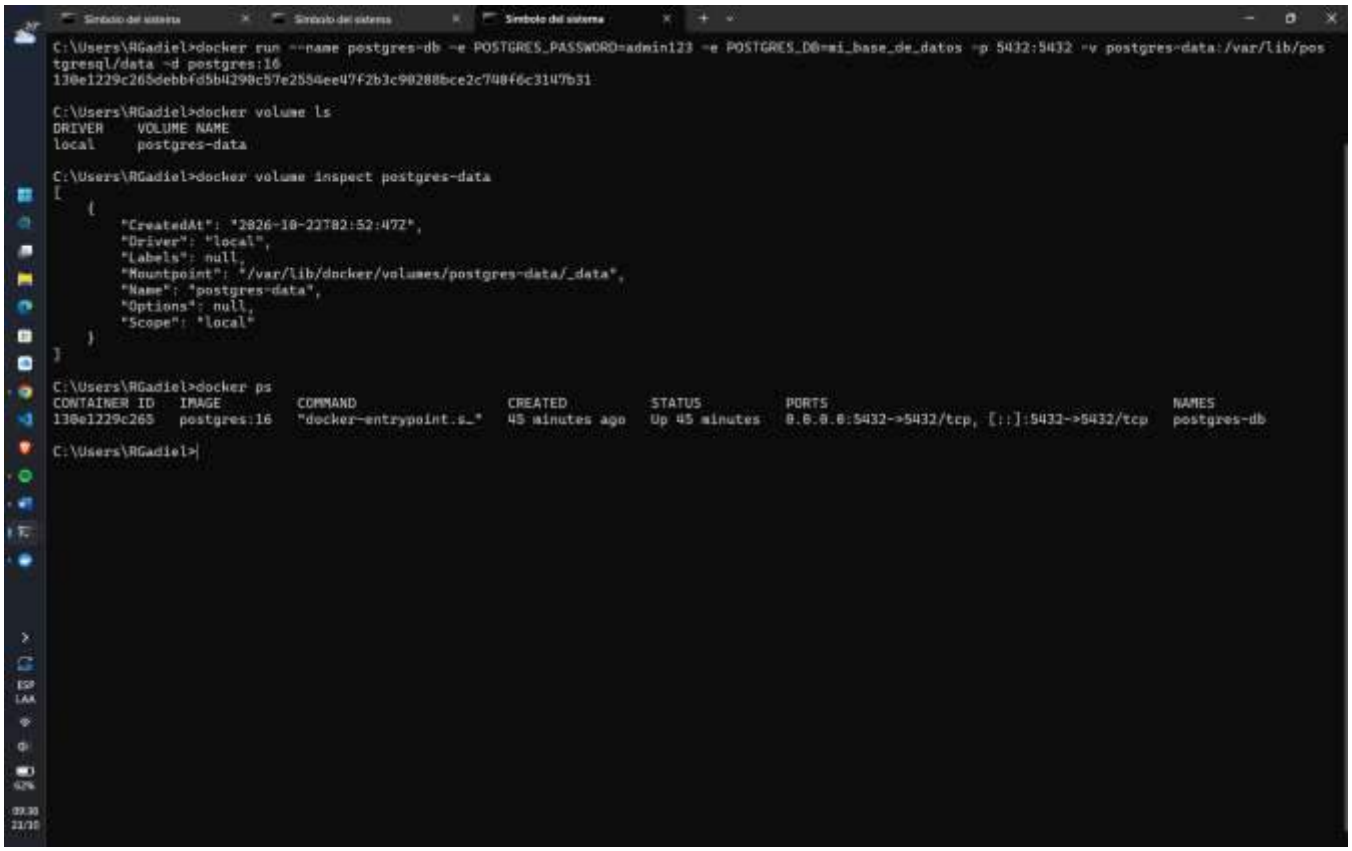
Imagen 12 Salida de docker volume ls y docker volume inspect postgres-data

- docker volume ls permite comprobar que el volumen existe.
- docker volume inspect postgres-data permite consultar sus metadatos reales, incluyendo la ruta donde Docker almacena los datos en el sistema anfitrión.
- Eliminar el contenedor no elimina automáticamente el volumen.
- Eliminar explícitamente el volumen (con "docker volume rm") sí elimina de forma permanente los datos almacenados en él.

Comprobación de PostgreSQL

Paso 1. Verificar que el contenedor está en ejecución, tanto desde la terminal como desde el panel "Containers" de Docker Desktop:

```
docker ps
```



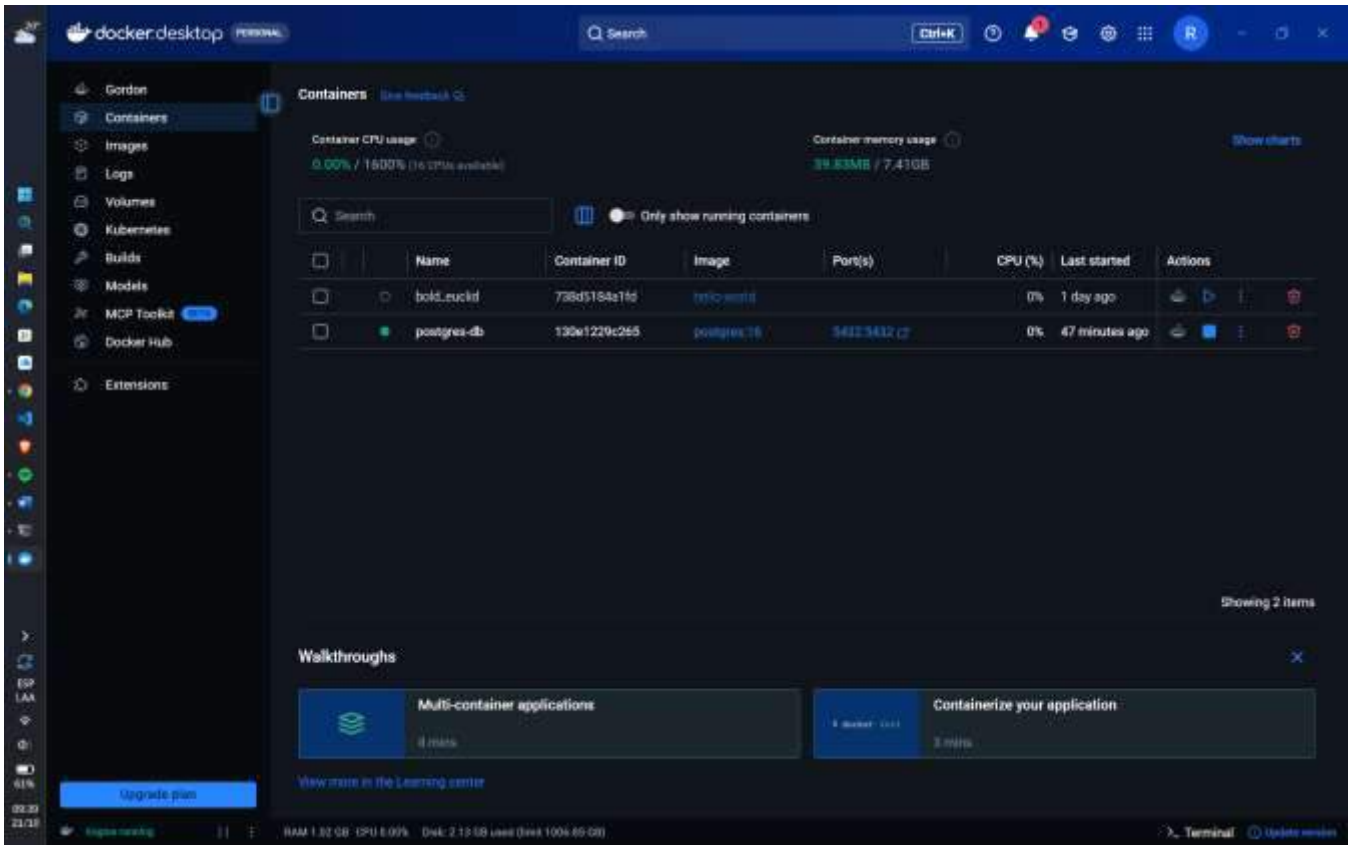


Imagen 13 Panel Containers de Docker Desktop y salida de docker ps, con postgres-db activo

Paso 2. Revisar el registro de arranque del servidor:

```
docker logs postgres-db
```

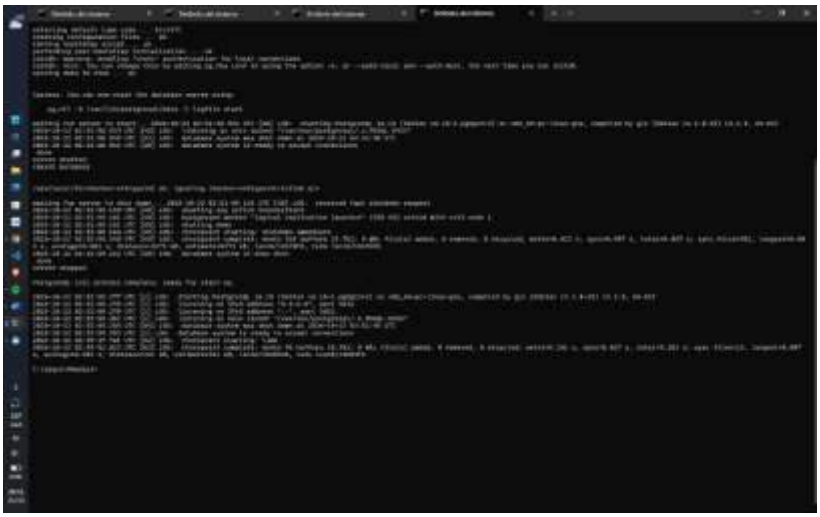


Imagen 14 Registro de arranque de PostgreSQL (docker logs postgres-db)

- **Qué debe observarse:** La inicialización del clúster de PostgreSQL 16.15, la creación de la base de datos y la línea "database system is ready to accept connections", repetida tras un reinicio del servidor dentro del mismo registro.
- **Resultado esperado:** La evidencia principal de que el arranque fue exitoso es la aparición de la línea "database system is ready to accept connections".

Detención y reinicio del contenedor

Es común detener el contenedor al finalizar una sesión de trabajo y volver a iniciarlo después. Detener el contenedor no elimina los datos, gracias al volumen previamente configurado.

```
docker stop postgres-db
docker start postgres-db
```



Imagen 15 Detención y reinicio del contenedor postgres-db

- **Qué debe observarse:** Ambos comandos devuelven el nombre del contenedor ("postgres-db") como confirmación.

Docker Compose

Método B: Docker Compose

Docker Compose es una herramienta incluida con Docker Desktop que permite definir, en un único archivo de texto ("docker-compose.yml"), la configuración completa de uno o varios contenedores. En esta práctica se utiliza como una forma alternativa al método basado en "docker run" descrito en el capítulo 6. No deben mantenerse simultáneamente dos contenedores con el mismo nombre o el mismo puerto publicado; por ello, antes de administrar el servicio mediante Compose, se eliminó por completo el entorno creado en pruebas anteriores.

Preparación del directorio de trabajo

Antes de recrear el entorno, se eliminaron los contenedores, los volúmenes y la red creados previamente, para partir de un estado limpio:

```
docker stop postgres-db pgadmin
docker rm postgres-db pgadmin

docker volume rm postgres-data pgadmin-data
docker network rm postgres-network
```

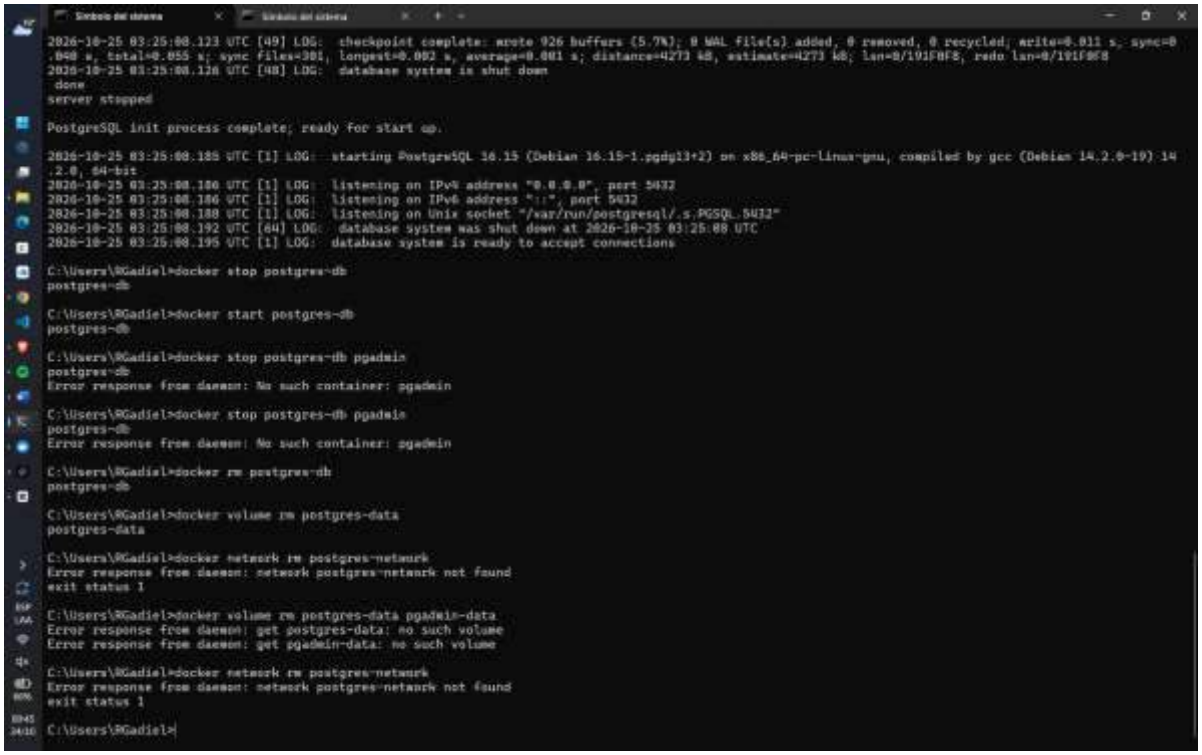


Imagen 16 Eliminación de los contenedores, volúmenes y red creados en pruebas anteriores

Con el entorno limpio, se creó nuevamente la carpeta de trabajo y se accedió a ella:

```
mkdir C:\proyectos\postgres-docker
cd C:\proyectos\postgres-docker
```

Se comprobó que la carpeta estuviera vacía antes de crear el archivo de configuración:

```
dir
```

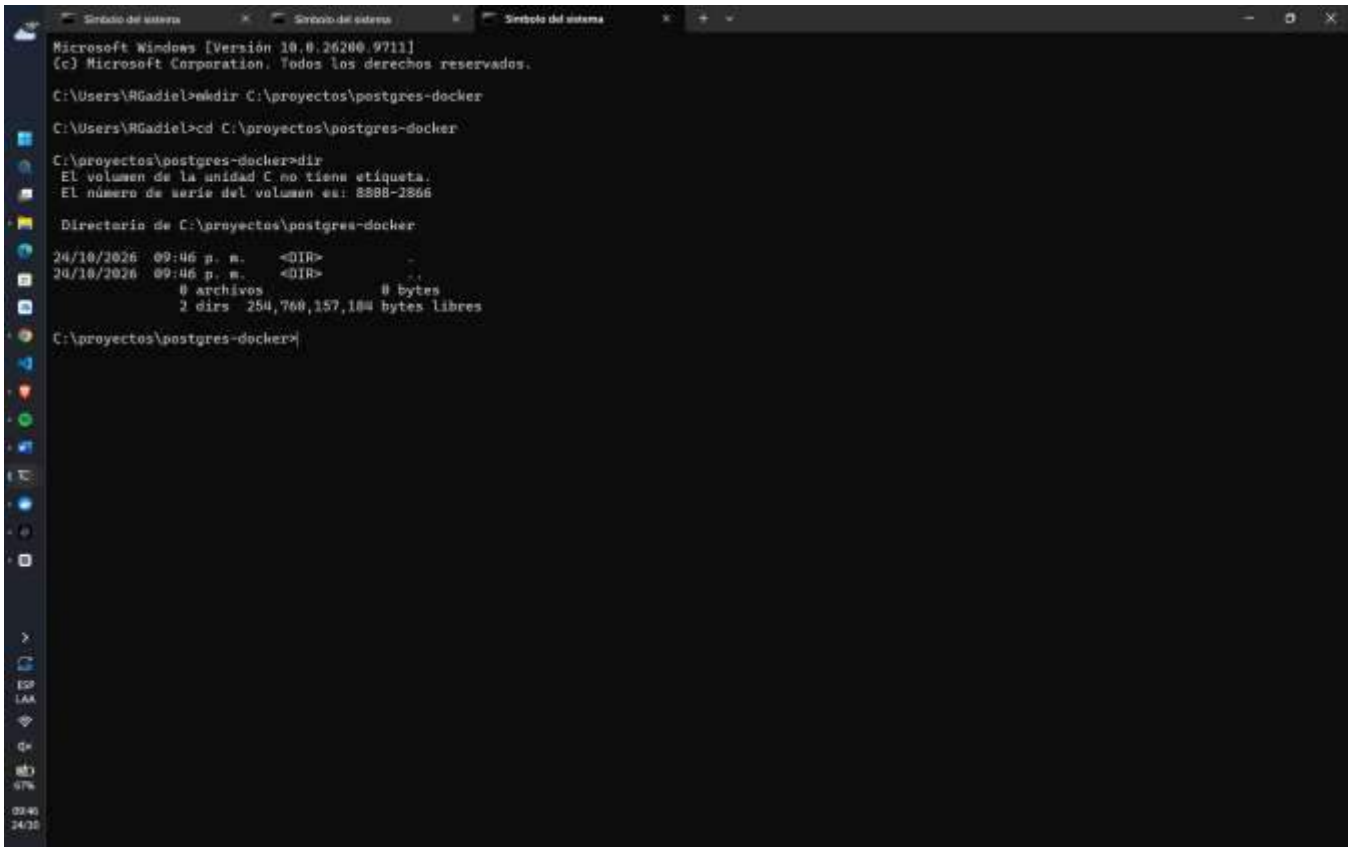


Imagen 17 Carpeta de trabajo creada y verificada como vacía

- **Qué debe observarse:** El resultado de "dir" mostrando "0 archivos" dentro de la carpeta, y la posterior apertura del editor mediante "code docker-compose.yml".
- **Resultado esperado:** La carpeta de trabajo existe y está vacía antes de crear el archivo docker-compose.yml

Creación del archivo docker-compose.yml

Dentro de esa carpeta se creó un archivo de texto llamado exactamente "docker-compose.yml", con el siguiente contenido, que define los servicios "postgres" y "pgadmin", sus variables de entorno, los puertos publicados, los volúmenes persistentes, la red compartida y una verificación de estado (healthcheck) para PostgreSQL:

```
services:


  postgres:

    image: postgres:16
    container_name: postgres-db
    restart: always


    environment:
      POSTGRES_USER: admin

      POSTGRES_PASSWORD: admin123

      POSTGRES_DB: mi_base_datos


    ports:

      - "5432:5432"


    volumes:

      - postgres-data:/var/lib/postgresql/data


    networks:

      - postgres-network
```

```
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U admin -d mi_base_datos"] interval: 10s
      timeout: 5s retries: 5


  pgadmin:
    image:          dpage/pgadmin4:latest
    container_name:  pgadmin  restart:
    always

    environment:
      PGADMIN_DEFAULT_EMAIL:                              admin@admin.com
      PGADMIN_DEFAULT_PASSWORD: admin123

    ports:
      - "8080:80"

    volumes:
      - pgadmin-data:/var/lib/pgadmin

    networks:
      - postgres-network

    depends_on:
      postgres:
        condition: service_healthy


volumes:    postgres-

  data:
    name: postgres-data

  pgadmin-data:
    name: pgadmin-data


networks:

  postgres-network:
    name: postgres-network driver: bridge
```

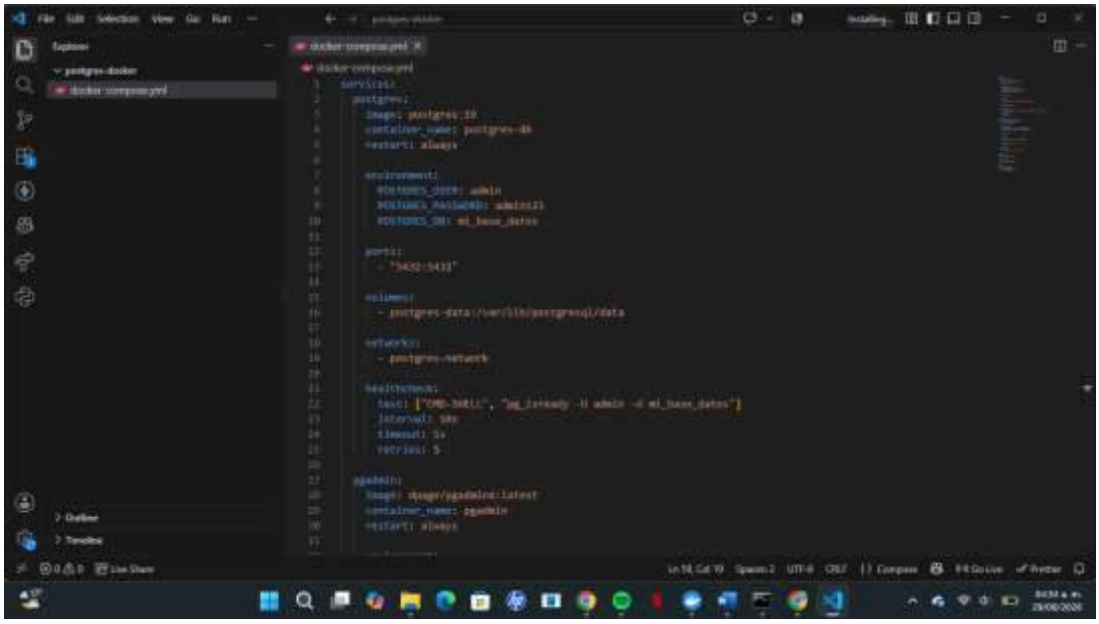


Imagen 18 Contenido del archivo docker-compose.yml en el editor

Las partes más relevantes de la configuración son las siguientes:

- postgres:16: versión de la imagen oficial de PostgreSQL utilizada para el servicio de base de datos.
- dpage/pgadmin4:latest: imagen oficial de pgAdmin 4 utilizada para administrar PostgreSQL desde el navegador.
- 5432:5432: publica el puerto de PostgreSQL en el equipo anfitrión.
- 8080:80: publica la interfaz web de pgAdmin (puerto 80 dentro del contenedor) en el puerto 8080 del equipo anfitrión.
- postgres-data: volumen con nombre que conserva los datos de PostgreSQL de forma persistente.
- pgadmin-data: volumen con nombre que conserva la configuración de pgAdmin de forma persistente.
- /var/lib/postgresql/data: ruta dentro del contenedor donde PostgreSQL almacena sus datos, asociada al volumen postgres-data.
- /var/lib/pgadmin: ruta dentro del contenedor donde pgAdmin guarda su configuración, asociada al volumen pgadmin-data.
- postgres-network: red de Docker declarada para que los contenedores postgres y pgadmin se comuniquen entre sí por nombre.
- healthcheck: comprobación periódica, mediante "pg_isready", que determina cuándo PostgreSQL queda disponible.
- depends_on con condition: service_healthy: obliga a que el contenedor pgadmin espere a que postgres reporte estado saludable antes de iniciar.

Validación del archivo YAML

Antes de levantar los servicios, se comprobó que la configuración fuera válida:

```
docker compose config
```

Este comando interpreta el archivo docker-compose.yml y muestra la configuración resuelta, lo que permite detectar errores de sintaxis o de indentación antes de crear los contenedores.

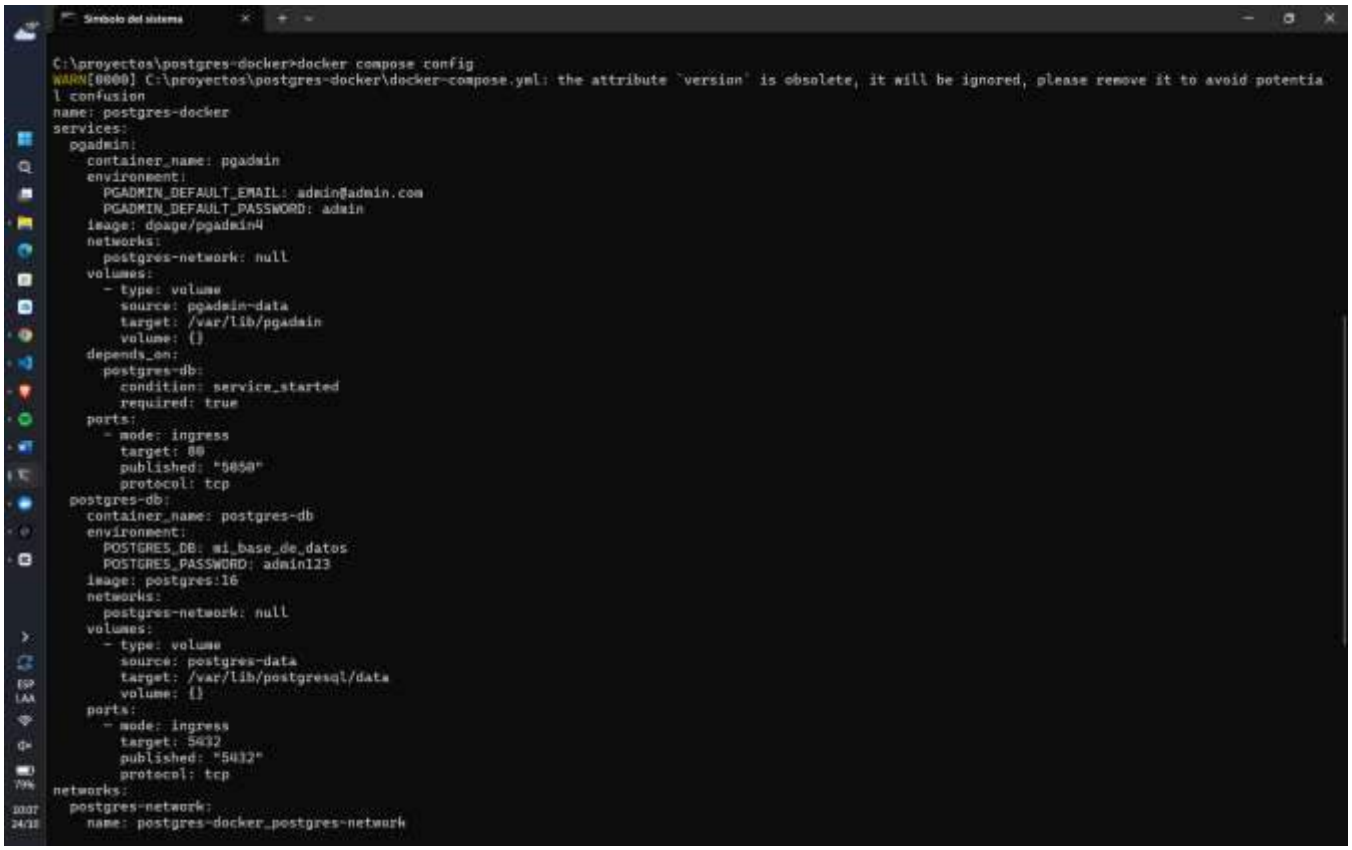


Imagen 19 Verificación del archivo creado y validación con docker compose config

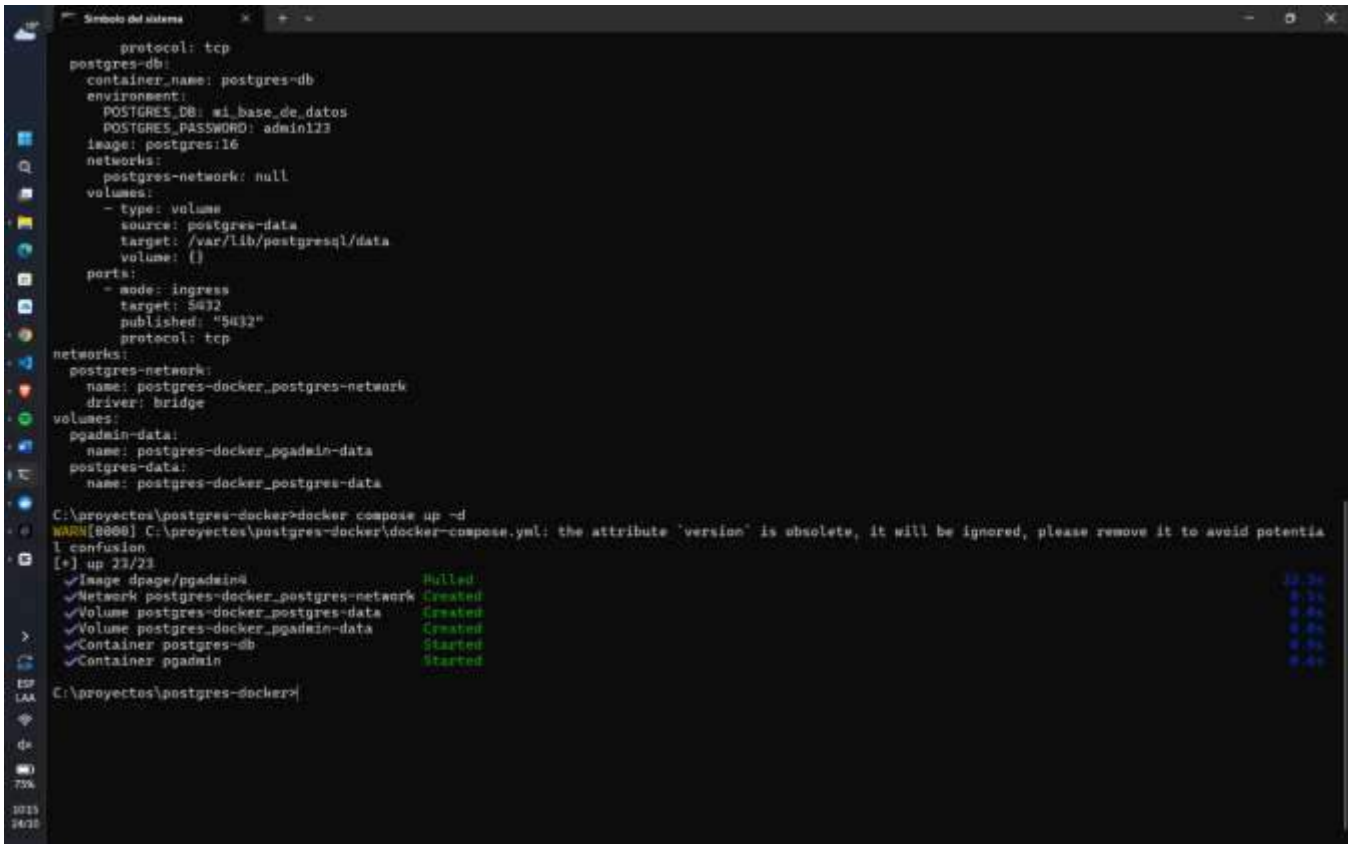
Creación y ejecución de los servicios

Con la configuración validada, se levantaron los servicios definidos en el archivo:

```
docker compose up -d
```

Docker Compose crea automáticamente la red, los volúmenes y los contenedores definidos en el archivo YAML, y a continuación se verificó su estado:

```
docker compose ps
```



Verificación de los contenedores

Se verificó nuevamente el estado de los servicios, junto con los volúmenes y la disponibilidad de PostgreSQL:

```
docker compose ps
docker ps

docker volume ls
```

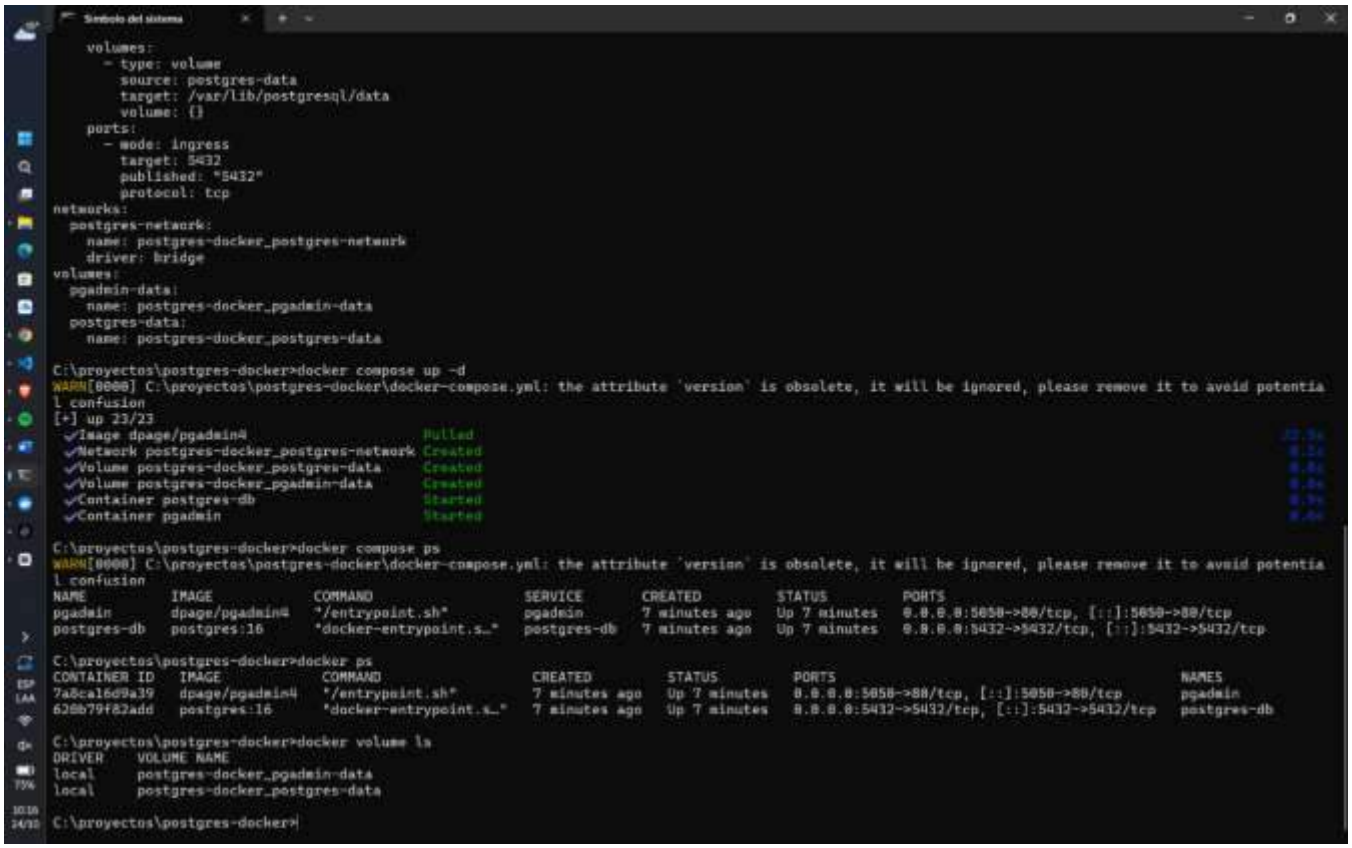


Imagen 21 Verificación de los contenedores, los volúmenes y la disponibilidad de PostgreSQL

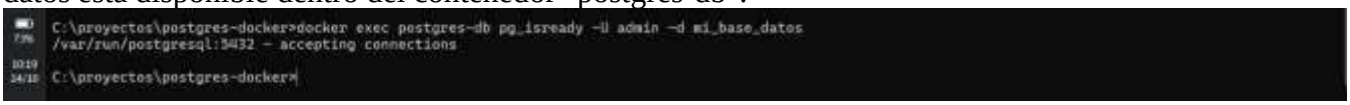
- **Qué debe observarse:** En "docker compose ps" / "docker ps", los servicios "postgres-db" (imagen postgres:16, puerto 0.0.0.0:5432->5432/tcp) y "pgadmin" (imagen dpage/pgadmin4:latest, puerto 0.0.0.0:8080->80/tcp), este último en estado "(healthy)"; en "docker volume ls", los volúmenes "postgres-data" y "pgadmin-data"; y en la salida de "pg_isready", el mensaje "accepting connections".

Verificación de PostgreSQL

Se comprobó de forma específica que PostgreSQL aceptara conexiones:

```
docker exec postgres-db pg_isready -U admin -d mi_base_datos
```

El mensaje "accepting connections", visible en la Figura 22, confirma que el servicio de base de datos está disponible dentro del contenedor "postgres-db".



Verificación de volúmenes

Se confirmó la creación de los volúmenes con nombre utilizados por ambos servicios:

```
docker volume ls
```

Los volúmenes utilizados son "postgres-data" y "pgadmin-data" (visibles en la Figura 22). Estos permiten conservar, respectivamente, los datos de PostgreSQL y la configuración de pgAdmin aunque los contenedores se detengan o se eliminen.

Verificación de volúmenes

Se confirmó la creación de los volúmenes con nombre utilizados por ambos servicios:

```
docker volume ls
```

Los volúmenes utilizados son "postgres-data" y "pgadmin-data" (visibles en la Figura 22). Estos permiten conservar, respectivamente, los datos de PostgreSQL y la configuración de pgAdmin aunque los contenedores se detengan o se eliminen.

Acceso a pgAdmin Web

pgAdmin se utiliza desde el navegador y se publica en el puerto 8080 del equipo anfitrión, definido en el archivo docker-compose.yml ("8080:80"). El acceso se realiza mediante:

```
http://localhost:8080
```

Las credenciales utilizadas en esta práctica son las declaradas en el archivo docker-compose.yml:

```
Correo: admin@admin.com
Contraseña: admin123
```

Se trata de credenciales de demostración y no deben utilizarse de esta forma en un entorno de producción.



Imagen 22 Verificación de la configuración y disponibilidad de los servicios

Configuración de la conexión PostgreSQL en pgAdmin

Dentro de pgAdmin se registra un nuevo servidor apuntando al contenedor "postgres-db", utilizando los datos reales de conexión definidos en el archivo docker-compose.yml:

- Name: PostgreSQL Docker
- Host name/address: postgres
- Port: 5432
- Maintenance database: mi_base_datos
- Username: admin

- Password: admin123

Es importante que el host de conexión sea "postgres" (el nombre del servicio definido en el archivo YAML) y no "localhost", ya que pgAdmin y PostgreSQL se ejecutan como contenedores independientes dentro de la misma red Docker ("postgres-network") y se alcanzan entre sí por el nombre del servicio, no por la dirección del equipo anfitrión.

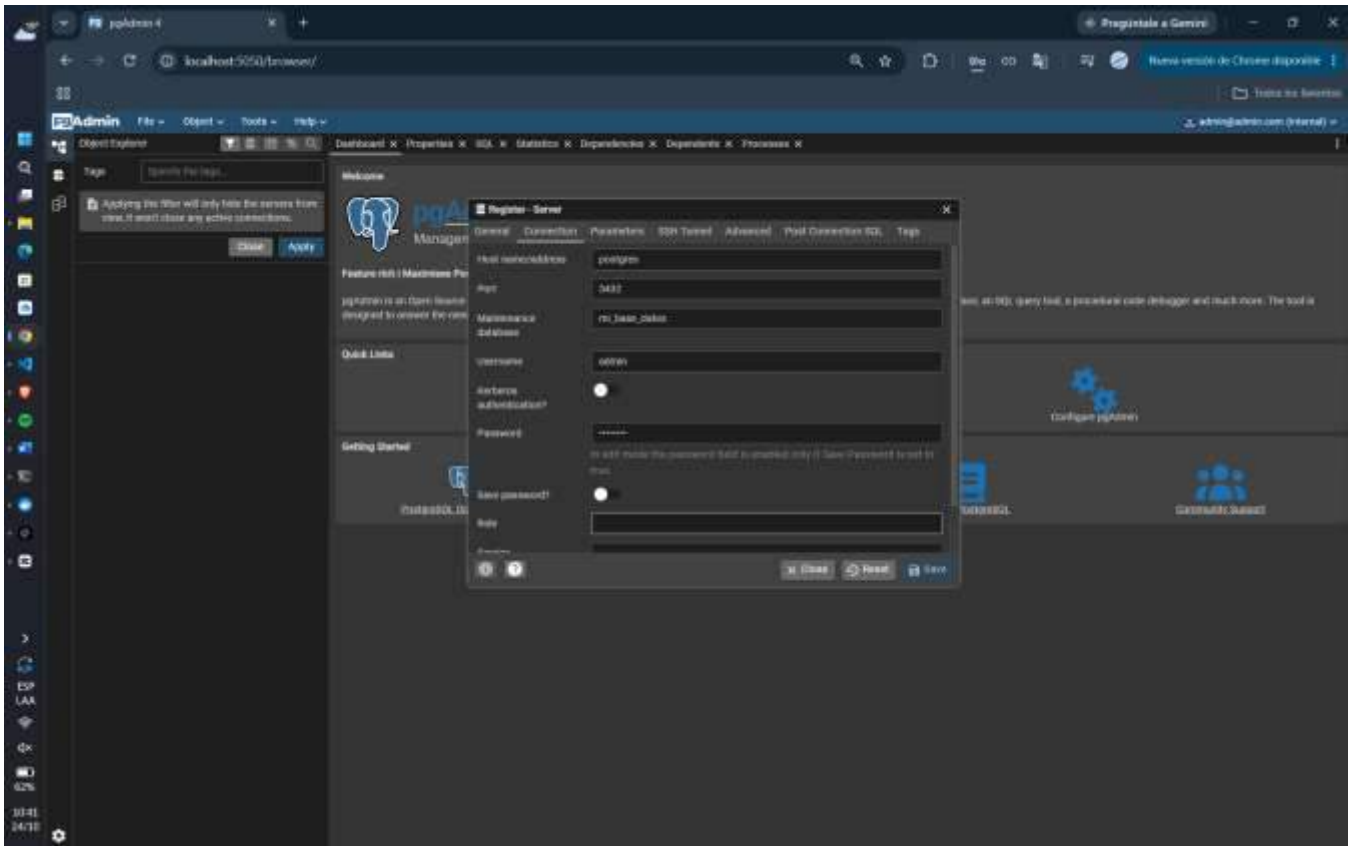


Imagen 23 Cuadro Register - Server de pgAdmin, pestaña Connection, con el host postgres, el puerto 5432, la base de mantenimiento mi_base_datos y el usuario admi

Persistencia de los datos

Es importante distinguir entre el contenedor y el volumen: el contenedor puede detenerse o eliminarse, mientras que los datos permanecen en el volumen con nombre "postgres-data", y la configuración de pgAdmin permanece en "pgadmin-data". Estos volúmenes existen de forma independiente al ciclo de vida de los contenedores.

Esta persistencia depende del ciclo de vida de los volúmenes, no del de los contenedores: el comando "docker compose down" detiene y elimina los contenedores y la red del proyecto, pero conserva los volúmenes "postgres-data" y "pgadmin-data" junto con los datos que contienen. Solo el comando "docker compose down -v" elimina también esos volúmenes y, con ellos, los datos almacenados.

Limpieza del entorno

Para detener y eliminar el entorno creado con Docker Compose se utiliza:

```
docker compose down
```

Este comando elimina los contenedores "postgres-db" y "pgadmin", así como la red "postgres-network" creada por Compose, pero no elimina los volúmenes "postgres-data" ni "pgadmin-data".

Si además se desea eliminar los volúmenes y, con ellos, los datos persistentes, se utiliza:

```
docker compose down -v
```

En este caso sí se eliminan los volúmenes "postgres-data" y "pgadmin-data", por lo que los datos almacenados en ellos se pierden de forma permanente.

Acceso externo a pgAdmin mediante Cloudflare Tunnel

Como complemento a la verificación del entorno, se realizó una prueba adicional para comprobar si la interfaz web de pgAdmin podía alcanzarse desde fuera de la red local en la que se encuentra el equipo anfitrión.

pgAdmin se publica mediante Docker en el puerto 8080 del equipo anfitrión. Para exponer temporalmente ese servicio hacia Internet, sin modificar la configuración del router ni asignar una dirección IP pública, se utilizó Cloudflare Tunnel (cloudflared), mediante un Quick Tunnel:

```
cloudflared tunnel --url http://localhost:8080
```

Este comando crea un túnel temporal sin necesidad de una cuenta de Cloudflare ni de un dominio propio, generando una dirección bajo el dominio trycloudflare.com que permanece activa únicamente mientras el proceso de cloudflared se mantiene en ejecución.

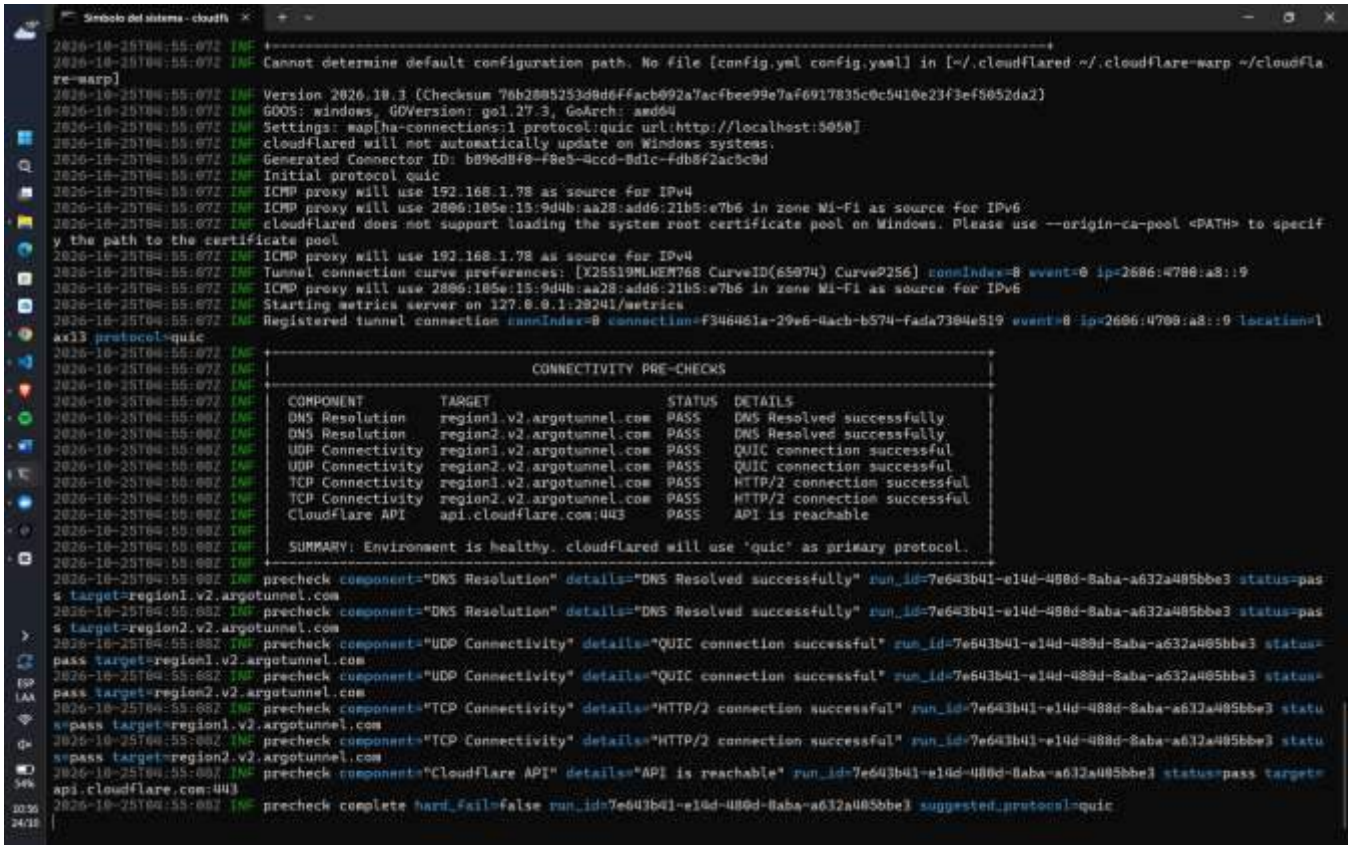


Imagen 24 Ejecución del Quick Tunnel de Cloudflare para pgAdmin

- **Qué debe observarse:** La salida de “docker ps” con pgAdmin publicado en el puerto 8080 y PostgreSQL en el puerto 5432, la versión de cloudflared, y la creación del Quick Tunnel con la dirección temporal generada bajo trycloudflare.com.
- **Resultado esperado:** El túnel se crea correctamente y queda a la espera de conexiones entrantes hacia el puerto 8080 del equipo anfitrión.

Una vez creado el túnel, se accedió a la dirección temporal generada por Cloudflare desde un dispositivo móvil conectado a una red distinta a la del equipo anfitrión, comprobando que la pantalla de inicio de sesión de pgAdmin se mostrara correctamente:



Figura 26. Acceso externo a pgAdmin desde una red diferente

> **Paso:** Después de crear el Quick Tunnel de Cloudflare.

Qué debe observarse: La pantalla de inicio de sesión de pgAdmin, mostrada en el navegador de un dispositivo móvil conectado a una red distinta, a través de la dirección temporal bajo trycloudflare.com.

Resultado esperado: La interfaz web de pgAdmin resulta accesible desde fuera de la red local mediante el túnel temporal.

Esta prueba confirma únicamente el acceso externo a la interfaz web de pgAdmin a través del túnel; no se realizó ni se documenta una prueba de acceso externo directo a PostgreSQL por el puerto 5432. La dirección generada por Cloudflare es temporal: se creó mediante un Quick Tunnel únicamente para esta prueba y dejó de estar disponible al cerrar el proceso de cloudflared, por lo que no debe interpretarse como una dirección permanente del sistem